

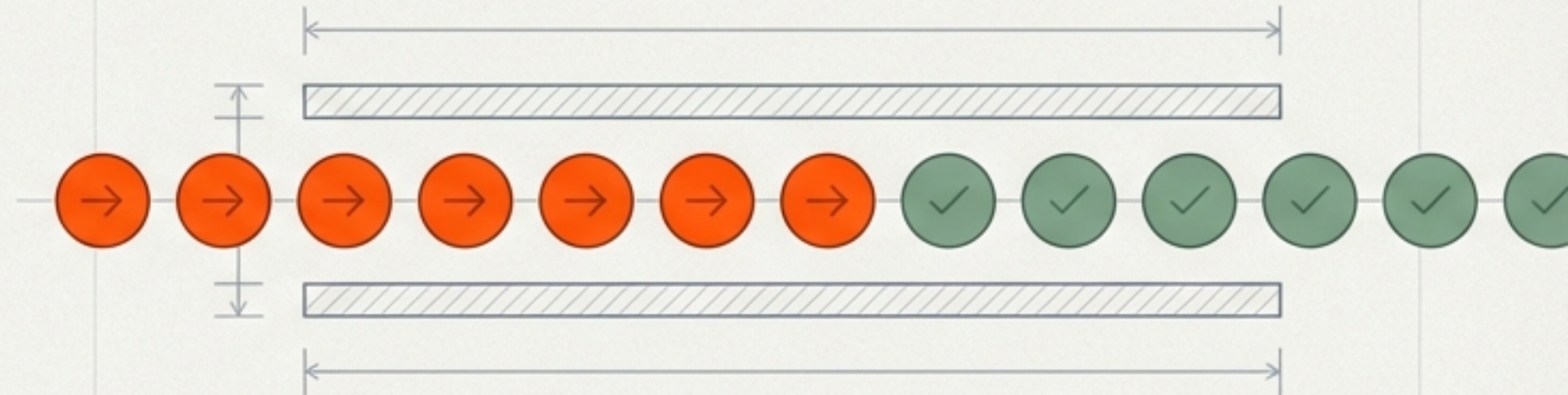
Inter Tight Queues & Deques in Python

JetBrains Mono

Mastering Data Structures
for APIs, System Architecture,
and Interviews.

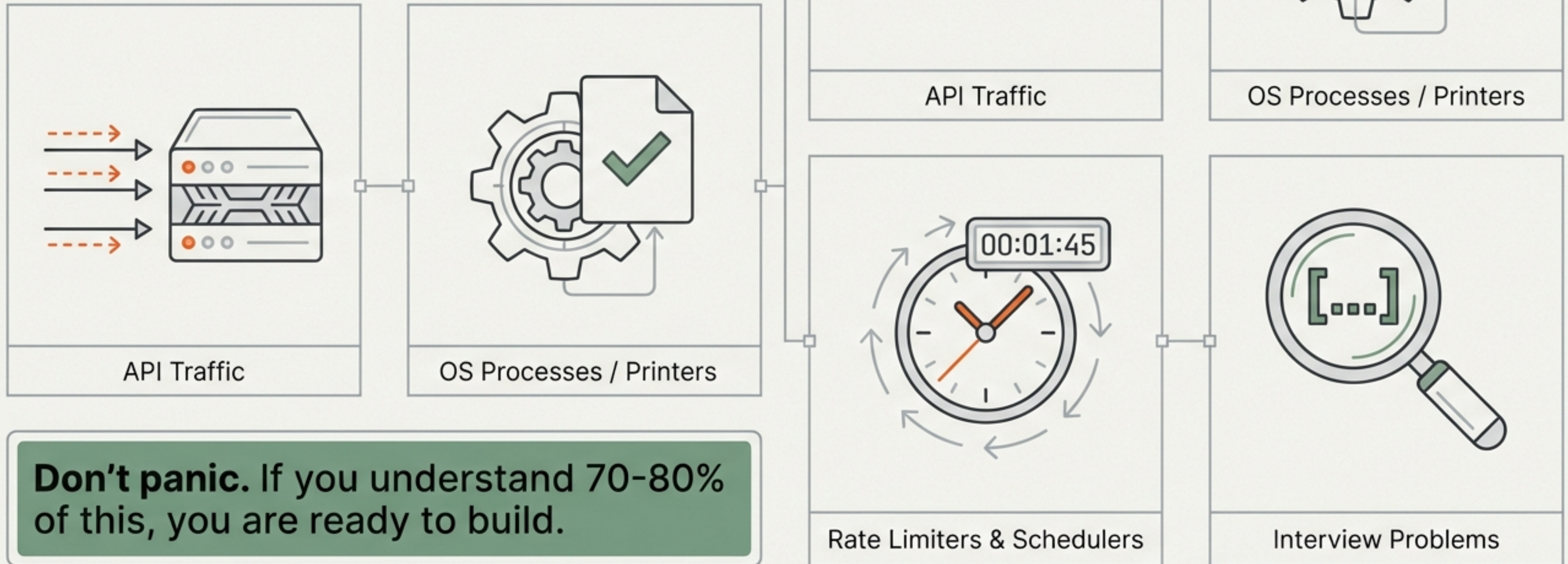
JetBrains Mono

Based on the 'Chai aur DSA' series.



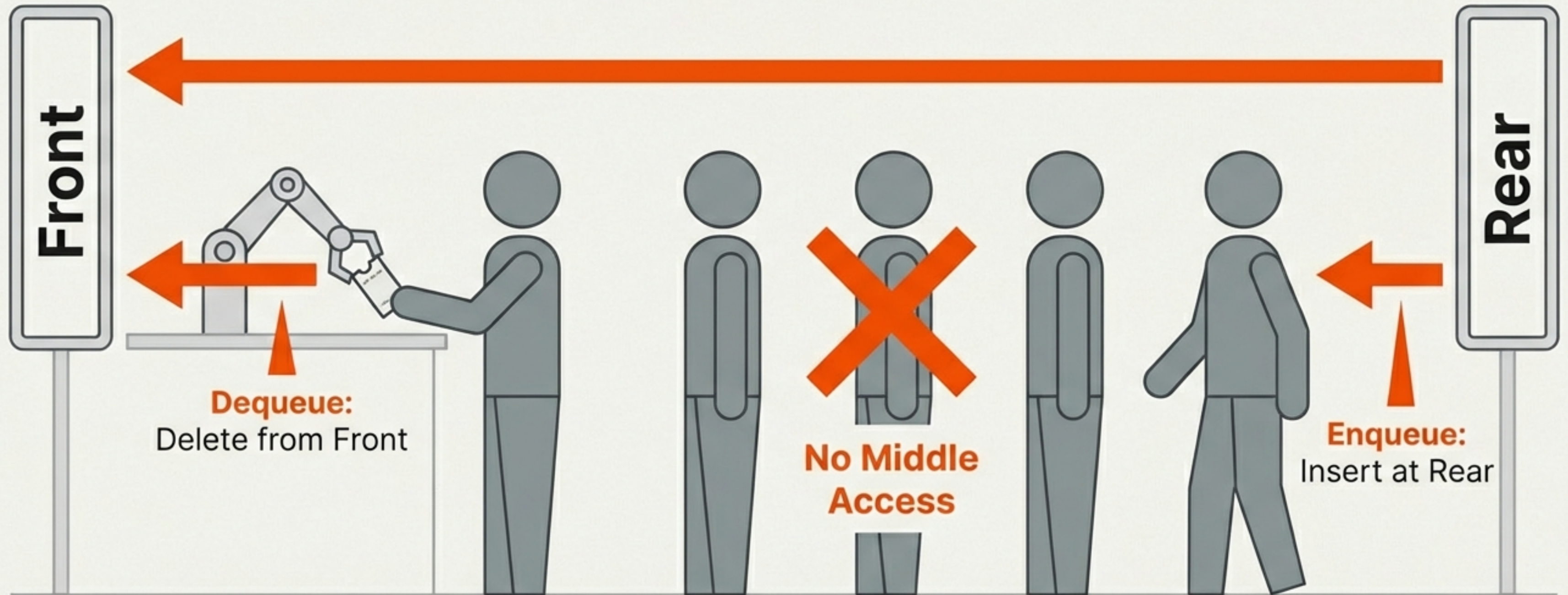
Everywhere, Unseen.

Queues are the backbone of modern system architecture. They aren't just for exams; they run the world.



The Rule is FIFO.

First In, First Out.



Under the Hood: The Traditional Approach

In languages like C/C++, you manage memory manually.

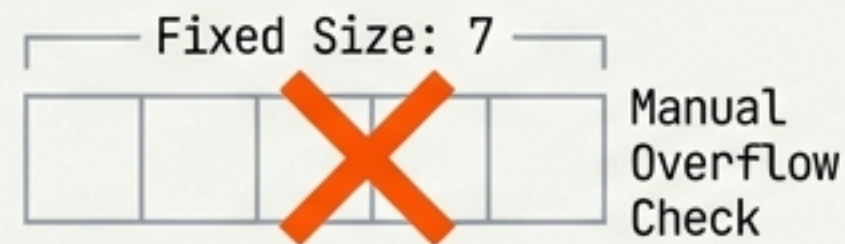


The Python Advantage: Dynamic Lists

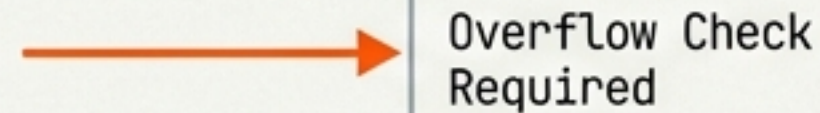
Python's List acts as a dynamic array, managing memory automatically.

Traditional Array

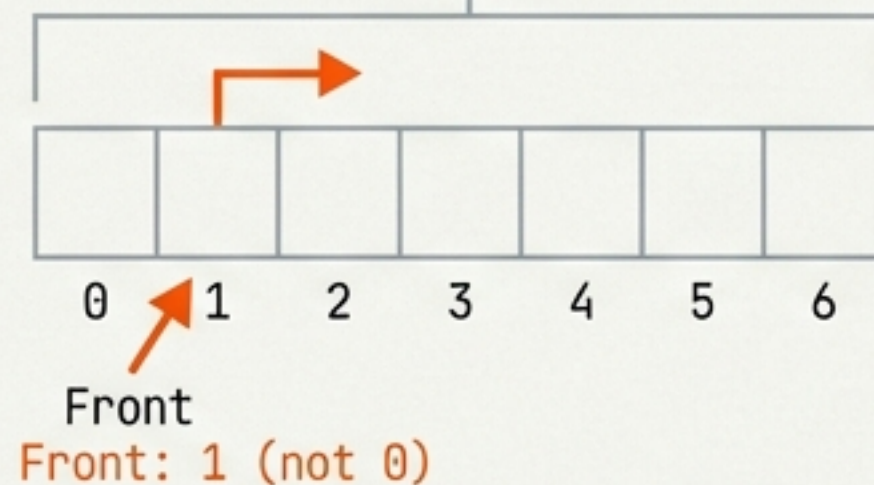
Fixed Size



Manual Overflow Check



Front pointer moves away from index 0

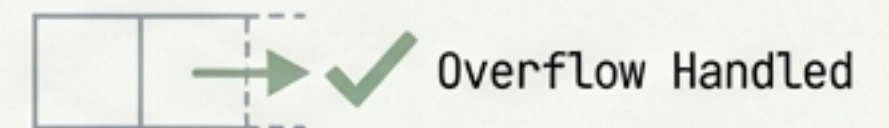


Python List

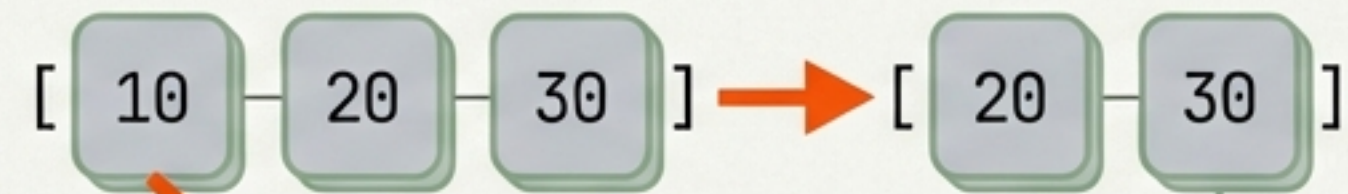
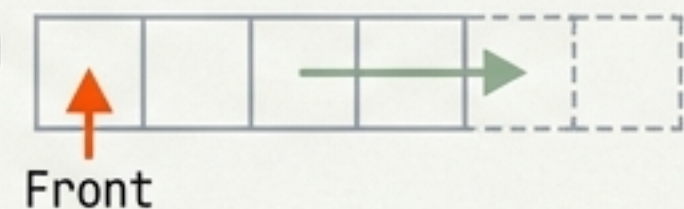
Grows Automatically



No Overflow Check



Front stays at index 0 (Auto-Shift)



Building the Queue Class

```
class Queue:
    def __init__(self):
        self.items = [] # Creates an empty list

    def is_empty(self):
        return len(self.items) == 0
```

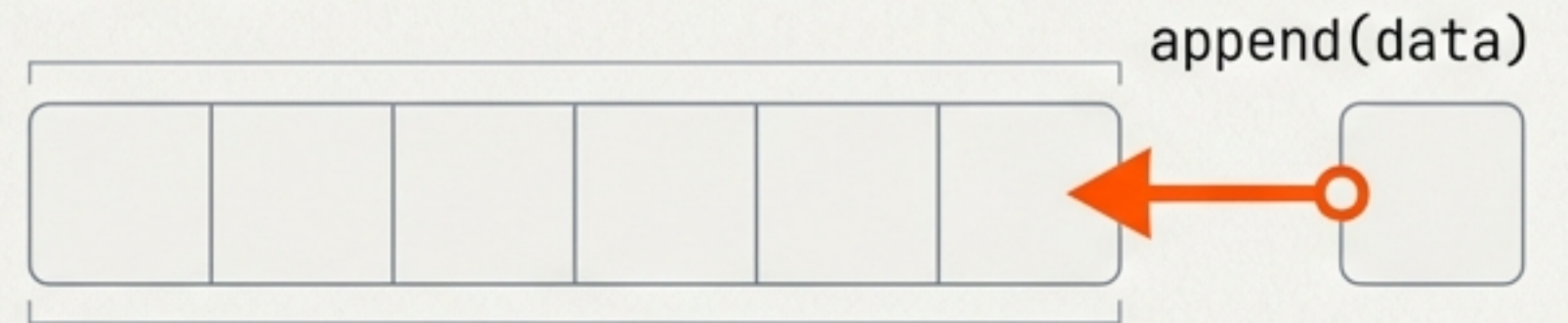
← Initializes empty
dynamic list

← Underflow
Protection Check

Operations: Enqueue & Dequeue

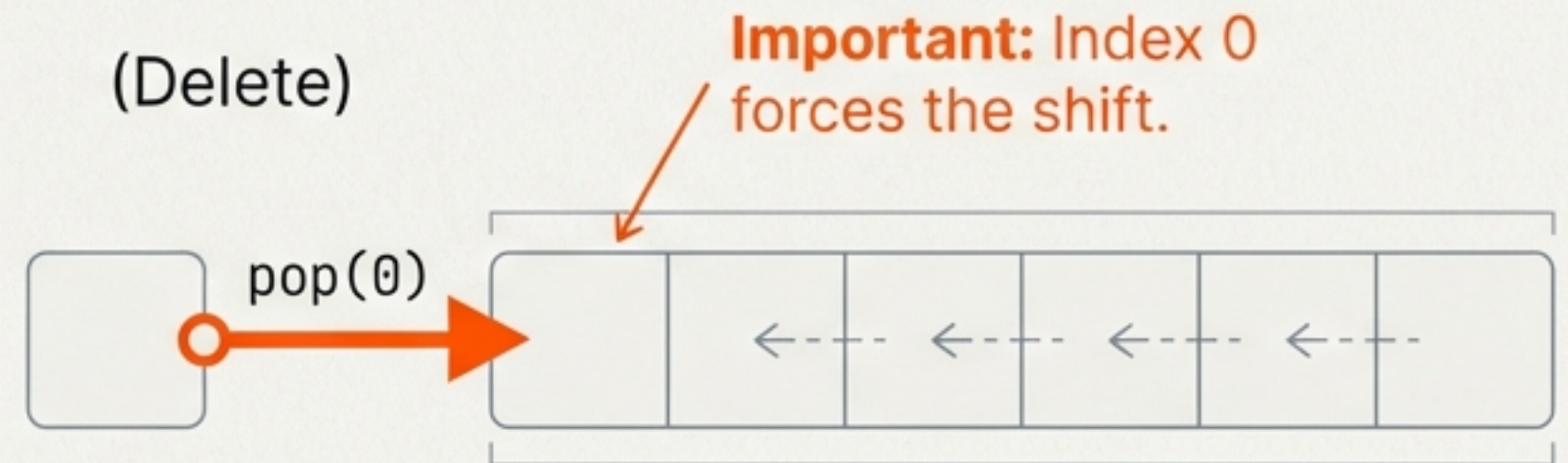
```
def insert(self, data):  
    self.items.append(data)  
  
def delete(self):  
    if not self.is_empty():  
        return self.items.pop(0)  
    else:  
        print('Queue is Empty')
```

(Insert)



List

(Delete)



List

Breaking the Rules: The Deque

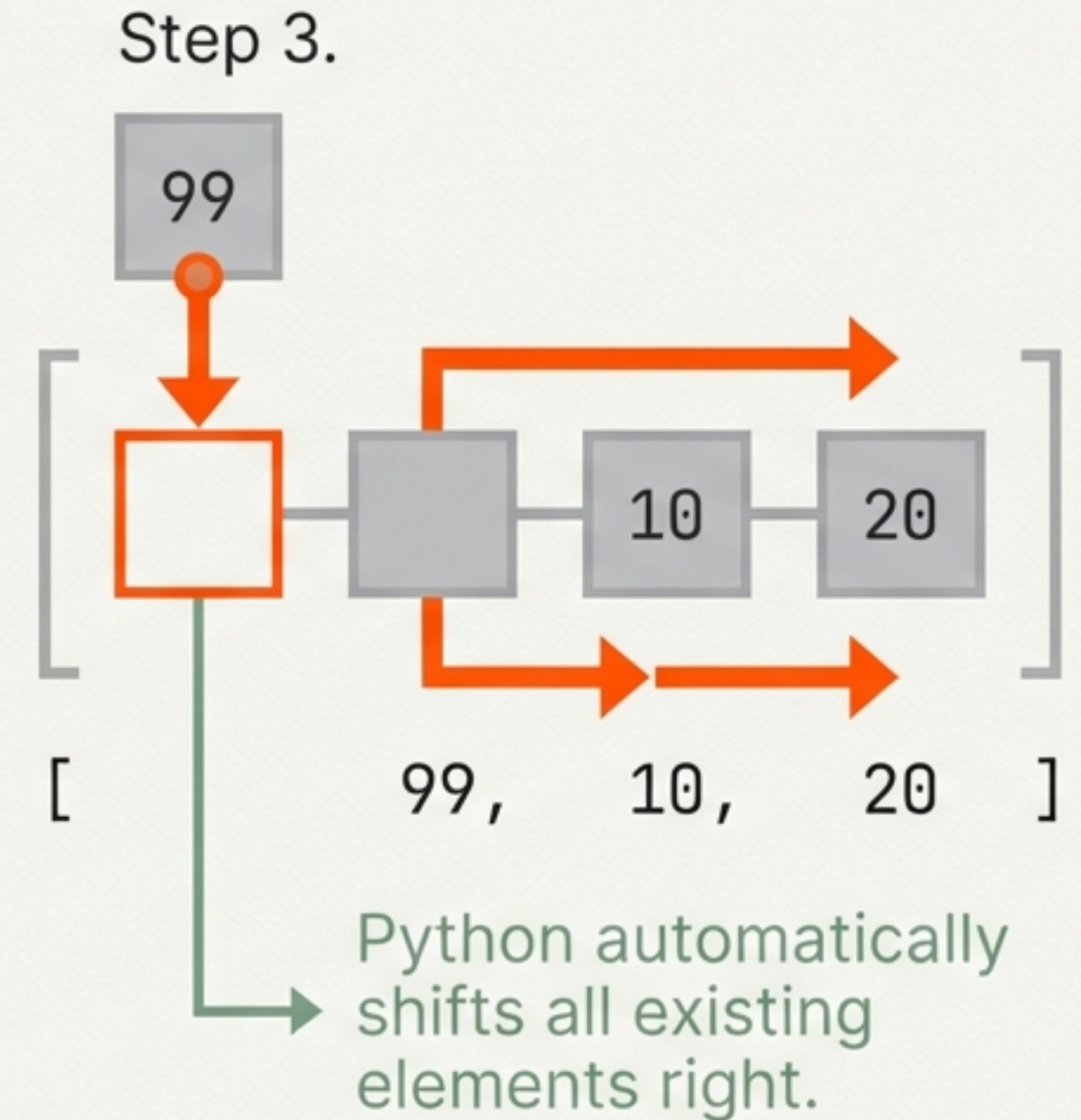
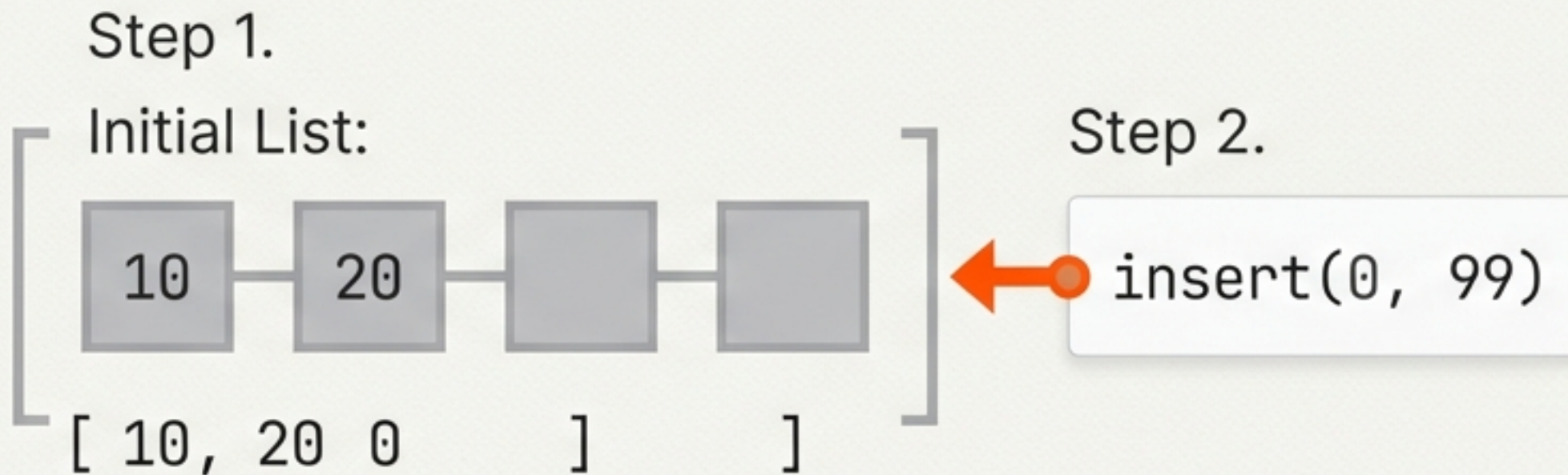
Pronounced 'Deck' (Double Ended Queue)



Unlike a standard Queue, a Deque allows two-way traffic at both ends.

The Challenge of 'Front' Insertion

Inserting at Index 0 requires making space.



Deque: Rear Operations

Identical to a standard Queue.

```
def insert_at_end(self, data):  
    self.items.append(data)
```

Adds to the END.

```
def delete_at_end(self):  
    if not self.is_empty():  
        return self.items.pop() # No index needed
```

Default pop() removes
the LAST item.
No shifting required.

Deque: Front Operations

Manipulating Index 0.

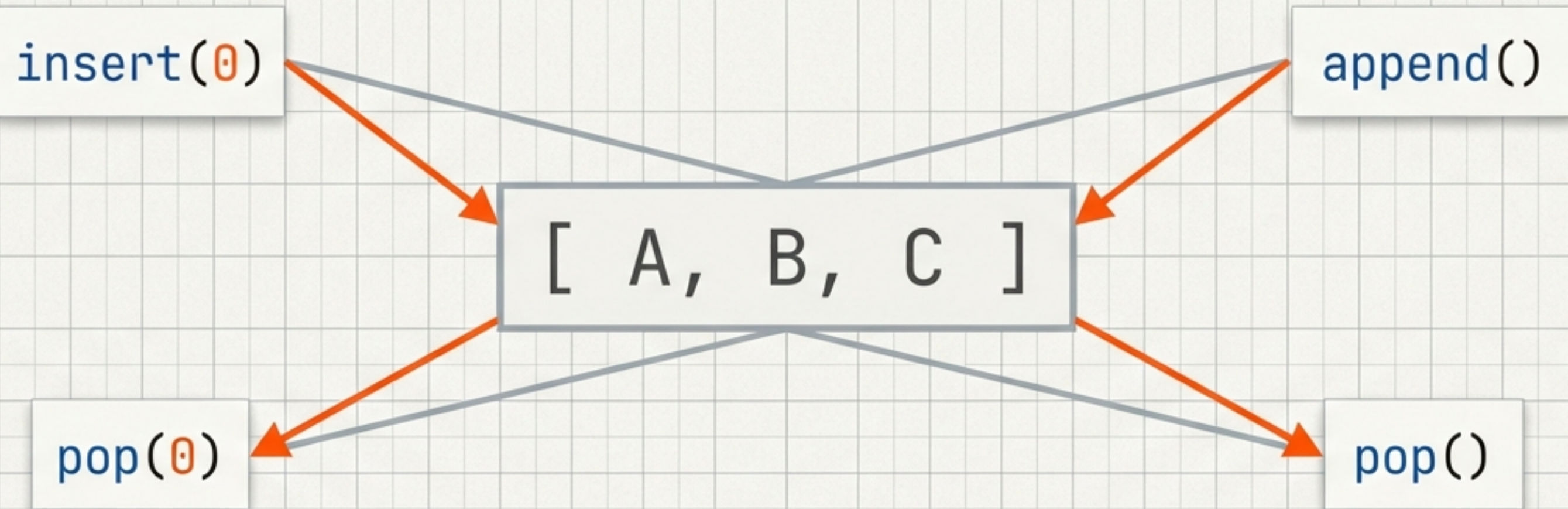
```
def insert_at_front(self, data):  
    self.items.insert(0, data) # Shifts Right
```

Targeting Index 0
forces right shift.

```
def delete_at_front(self):  
    if not self.is_empty():  
        return self.items.pop(0) # Shifts Left
```

Targeting Index 0
forces left shift.

The Complete Dequeue Class



A simple Python list acts as a powerful Double Ended Queue.

Don't Crash: Safety & Edge Cases



Underflow Error

You cannot pop from an empty list.

Always check ``is_empty()`` before deleting.

Middle Access

Just because Python allows ``list[2]``, doesn't mean you should use it. Queues and Deques restrict access to the ends.

If you have 0 rupees, you can't give someone 10.

Summary & Performance

Queue	FIFO	Insert Rear, Delete Front
Deque	Double Ended	Insert/Delete at Both Ends

Performance Note:

- Rear Operations: `append()` and `pop()` are $O(1)$ [Fast]
- Front Operations: `insert(0)` and `pop(0)` are $O(n)$ [Slower due to shifting]

For massive scale, use ``collections.deque``.

Practice Makes Perfect

1. Code this yourself. Don't just watch.
2. Try deleting elements until the Queue is empty to trigger the Underflow check.
3. Up Next: Circular Queues.

Master the list, and you master the flow of data.